

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

"C++ PROCESSES / THREADS"

Реквизиты

Дмитрий Малов, Z33433, 2025 год

Цель работы

Познакомить студента с принципами параллельных вычислений. Составить несколько программ в простейшими вычислительными действиями, чтобы освоить принципы параллельных вычислений (когда одни алгоритмы зависят / не зависят от других).

Задача 1

[C++ SEQUENCE] Последовательные вычисления

Требуется последовательно выполнить вычисления по формуле 1, вычисления по формуле 2, после чего выполнить вычисления по формуле 3, которые выглядят следующим образом: результат вычислений 1 + результат вычислений 2 – результат вычислений 1. Выполнить последовательно на 10 000 итераций и 100 000 итераций. Формула 1: $f(x) = x^2 - x^2 + x * 4 - x * 5 + x + x$ Формула 2: $f(x) = x + x$ Вывести длительность выполнения всех 10 000 итераций и 100 000 итераций в сек.

Задача 2

[C++ THREADS] Параллельные вычисления через потоки

Требуется параллельно (насколько возможно с помощью потоков) выполнить вычисления по формуле 1, вычисления по формуле 2, после чего выполнить вычисления по формуле 3, которые выглядят следующим образом: результат вычислений 1 + результат вычислений 2 – результат вычислений 1. Выполнить последовательно на 10 000 итераций и 100 000 итераций. Формула 1: $f(x) = x^2 - x^2 + x * 4 - x * 5 + x + x$ Формула 2: $f(x) = x + x$ Вывести длительность выполнения всех 10 000 итераций и 100 000 итераций в сек. в разбивке по шагам вычислений 1, 2 и 3

Задача 3

[C++ PROCESS] Параллельные вычисления через процессы

Требуется параллельно (насколько возможно с помощью процессов) выполнить вычисления по формуле 1, вычисления по формуле 2, после чего выполнить вычисления по формуле 3, которые выглядят следующим образом: результат вычислений 1 + результат вычислений 2 – результат вычислений 1. Выполнить последовательно на 10 000 итераций и 100 000 итераций. Формула 1: $f(x) = x^2 - x^2 + x * 4 - x * 5 + x + x$ Формула 2: $f(x) = x + x$ Вывести длительность выполнения всех 10 000 итераций и 100 000 итераций в сек. в разбивке по шагам вычислений 1, 2 и 3

Решение задачи 1

```
#include <iostream>
#include <cmath>
#include <chrono>
#include <cstdlib>

using namespace std;
using namespace std::chrono;

double formula1(double x) {
    return pow(x, 2) - pow(x, 2) + 4*x - 5*x + x + x;
}

double formula2(double x) {
    return x + x;
}

double formula3(double res1, double res2) {
    return res1 + res2 - res1;
}

void calculate(int iterations) {
    auto start = high_resolution_clock::now();

    for (int i = 1; i <= iterations; ++i) {
        int x = rand();
        double res1 = formula1(x);
        double res2 = formula2(x);
        double res3 = formula3(res1, res2);
    }

    auto stop = high_resolution_clock::now();
    duration<double> sum_time = stop - start;

    cout << "Iterations: " << iterations << ", Time: "
         << sum_time.count() << " seconds" << endl;
}

int main() {
    calculate(10000);
    calculate(100000);
    return 0;
}
```

Решение задачи 2

Логика решения следующая: сначала параллельно по потокам считаются формулы 1 и 2, каждый поток берет $N/N_{threads}$ вычислений. Потом снова параллельно по потокам считается формула 3 для уже найденных значений формул 1 и 2

```

#include <iostream>
#include <cmath>
#include <chrono>
#include <thread>
#include <vector>
#include <mutex>
#include <iomanip>
#include <cstdlib>

using namespace std;
using namespace std::chrono;

mutex cout_mutex;

double formulal(double x) {
    return pow(x, 2) - pow(x, 2) + 4*x - 5*x + x + x;
}

double formula2(double x) {
    return x + x;
}

double formula3(double res1, double res2) {
    return res1 + res2 - res1;
}

void compute_formulas_1_2(int start, int end, vector<double>& res1, vector<double>& res2) {
    for (int i = start; i <= end; ++i) {
        int x = rand();
        res1[i] = formulal(i+1);
        res2[i] = formula2(i+1);
    }
}

void compute_formula_3(int start, int end, const vector<double>& res1, const vector<double>& res2, double& time_accum) {
    auto start_time = high_resolution_clock::now();

    for (int i = start; i <= end; ++i) {
        double res = formula3(res1[i], res2[i]);
    }

    auto end_time = high_resolution_clock::now();
    double elapsed = duration_cast<microseconds>(end_time - start_time).count() / 1e6;

    lock_guard<mutex> lock(cout_mutex);
    time_accum += elapsed;
}

```

```

void benchmark(int iterations) {
    const int num_threads = thread::hardware_concurrency();
    vector<double> results1(iterations);
    vector<double> results2(iterations);

    cout << fixed << setprecision(7);

    // Общее время начала выполнения
    auto total_start = high_resolution_clock::now();

    // Этап 1: Параллельное вычисление формул 1 и 2
    auto stage1_start = high_resolution_clock::now();
    {
        vector<thread> workers;
        int chunk_size = iterations / num_threads;

        for (int i = 0; i < num_threads; ++i) {
            int start = i * chunk_size;
            int end;
            if (i == num_threads - 1) {
                end = iterations - 1;
            } else {
                end = start + chunk_size - 1;
            }
            workers.emplace_back(compute_formulas_1_2, start, end, ref(results1), ref(results2));
        }

        for (auto& t : workers) {
            t.join();
        }
    }
    auto stage1_end = high_resolution_clock::now();
}

```

```

// Этап 2: Параллельное вычисление формулы 3
double stage2_time = 0;
auto stage2_start = high_resolution_clock::now();
{
    vector<thread> workers;
    int chunk_size = iterations / num_threads;

    for (int i = 0; i < num_threads; ++i) {
        int start = i * chunk_size;
        int end;
        if (i == num_threads - 1) {
            end = iterations - 1;
        } else {
            end = start + chunk_size - 1;
        }
        workers.emplace_back(compute_formula_3, start, end, cref(results1), cref(results2), ref(stage2_time));
    }

    for (auto& t : workers) {
        t.join();
    }
}
auto stage2_end = high_resolution_clock::now();

// Расчет временных показателей
double total_elapsed = duration_cast<microseconds>(stage2_end - total_start).count() / 1e6;
double stage1_elapsed = duration_cast<microseconds>(stage1_end - stage1_start).count() / 1e6;
double stage2_elapsed = duration_cast<microseconds>(stage2_end - stage2_start).count() / 1e6;

// Вывод результатов
lock_guard<mutex> lock(cout_mutex);
cout << "=== Iterations: " << iterations << " ===" << endl;
cout << "Threads used: " << num_threads << endl;
cout << "Total time: " << total_elapsed << " sec" << endl;
cout << "Stage 1 (F1+F2) time: " << stage1_elapsed << " sec" << endl;
cout << "Stage 2 (F3) time: " << stage2_elapsed << " sec" << endl;
cout << "Sum stages: " << (stage1_elapsed + stage2_elapsed) << " sec" << endl;
cout << "-----" << endl;
}

int main() {
    benchmark(1000);
    benchmark(10000);
    benchmark(100000);
    benchmark(1000000);

    return 0;
}

```

Решение задачи 3

```
#include <iostream>
#include <cmath>
#include <chrono>
#include <vector>
#include <unistd.h>
#include <sys/wait.h>
#include <iomanip>

using namespace std;
using namespace std::chrono;

struct ProcessResults {
    double time1;
    double time2;
    double time3;
};

double formulal(int x) {
    return pow(x, 2) - pow(x, 2) + 4*x - 5*x + x + x;
}

double formula2(int x) {
    return x + x;
}

double formula3(double res1, double res2) {
    return res1 + res2 - res1;
}

void child_work(int start, int end, ProcessResults* results) {

    auto t1_start = steady_clock::now();
    for (int i = start; i <= end; ++i) {
        volatile double res = formulal(i);
        (void)res;
    }
    results->time1 = duration_cast<microseconds>(steady_clock::now() - t1_start).count() / 1e6;

    auto t2_start = steady_clock::now();
    for (int i = start; i <= end; ++i) {
        volatile double res = formula2(i);
        (void)res;
    }
    results->time2 = duration_cast<microseconds>(steady_clock::now() - t2_start).count() / 1e6;
```

```

    auto t3_start = steady_clock::now();
    for (int i = start; i <= end; ++i) {
        volatile double res = formula3(formula1(i), formula2(i));
        (void)res;
    }
    results->time3 = duration_cast<microseconds>(steady_clock::now() - t3_start).count() / 1e6;
}

void calculate(int iterations) {
    const int num_processes = sysconf(_SC_NPROCESSORS_ONLN);
    vector<ProcessResults> results(num_processes);
    vector<pid_t> pids(num_processes);

    auto total_start = steady_clock::now();

    int chunk = iterations / num_processes;
    for (int i = 0; i < num_processes; ++i) {
        int start = i * chunk + 1;
        int end = (i == num_processes - 1) ? iterations : (i + 1) * chunk;

        pid_t pid = fork();
        if (pid == 0) {
            child_work(start, end, &results[i]);
            exit(0);
        }
        pids[i] = pid;
    }

    for (pid_t pid : pids) {
        waitpid(pid, nullptr, 0);
    }

    double total_time = duration_cast<microseconds>(steady_clock::now() - total_start).count() / 1e6;

    double sum1 = 0, sum2 = 0, sum3 = 0;
    for (const auto& res : results) {
        sum1 += res.time1;
        sum2 += res.time2;
        sum3 += res.time3;
    }

    cout << fixed << setprecision(6);
    cout << "=== Iterations: " << iterations << " ===" << endl;
    cout << "Processes: " << num_processes << endl;
    cout << "Total time: " << total_time << " sec" << endl;
    cout << "Formula1 total: " << sum1 << " sec" << endl;
    cout << "Formula2 total: " << sum2 << " sec" << endl;
    cout << "Formula3 total: " << sum3 << " sec" << endl;
    cout << "-----" << endl;
}

```

Заключение

- Результаты последовательной программы:

```
Iterations: 10000, Time: 0.0010071 seconds
Iterations: 100000, Time: 0.0069916 seconds
```

- Результаты параллельной по потокам программы:

```
=== Iterations: 1000 ===
Threads used: 8
Total time: 0.0020060 sec
-----
=== Iterations: 10000 ===
Threads used: 8
Total time: 0.0009990 sec
-----
=== Iterations: 100000 ===
Threads used: 8
Total time: 0.0030000 sec
-----
=== Iterations: 1000000 ===
Threads used: 8
Total time: 0.0160220 sec
-----
```

- Результаты параллельной по процессам программы:

```
=== Iterations: 1000 ===
Processes: 8
Total time: 0.001633 sec
-----
=== Iterations: 10000 ===
Processes: 8
Total time: 0.001564 sec
-----
=== Iterations: 100000 ===
Processes: 8
Total time: 0.002841 sec
-----
=== Iterations: 1000000 ===
Processes: 8
Total time: 0.018796 sec
-----
```

- Интерпретация результатов:

Для 10000 итераций наименее эффективны процессы, по-видимому, их запуск стоит дороже, чем выигрыш, который они дают.

Для 100000 итераций параллельные реализации дают выигрыш по времени